

— TRAINING-FREE · NO TASK HEAD · OPEN VOCABULARY

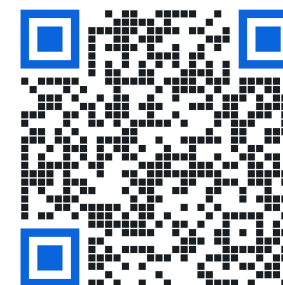
Test-time compute of jina-embeddings-v5-omni for image tagging

Han Xiao

VP of AI, Elastic

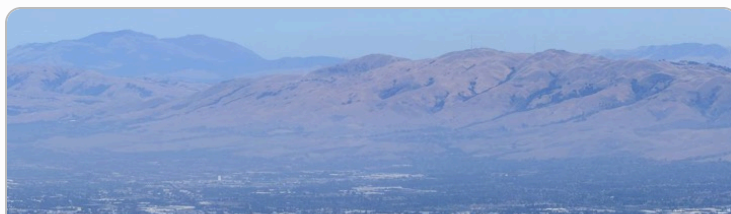
[@hxiao](#) · [in/hxiao87](#)

THIS DECK

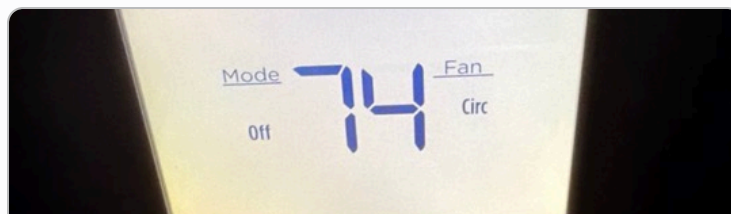


THE RESULT, FIRST

Training-free, open-vocabulary tagging from a frozen embedding model.



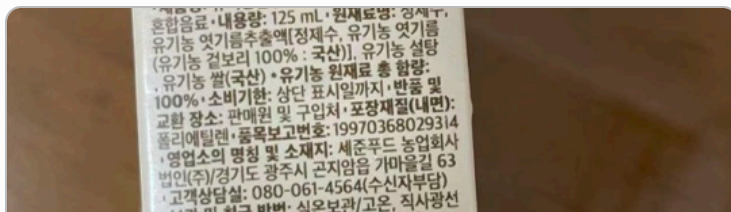
mountain · phoenix · sky · rooftop · location



clock · thermostat · dial · dst · wifi



fiyat · pagamento · brasile · gratuiti · utils



contiene · liters · dosage · thirst · consumo



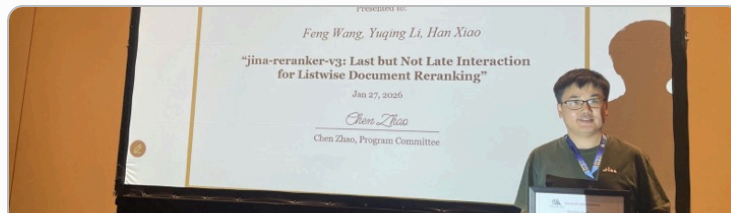
harga · lcd · samsung · refurbished · warrants



sunglasses · watches · branded · bronze · necklace



tablet · motherboard · wearable · celular · datastore



award · forwarding · speaking · defendant · workshop



gift · toy · celebration · balloons · cheerful

Problem formulation.

- | GIVEN one embedding model, **jina-embeddings-v5-omni-nano** (~1B, image and text in one 768-d space), and one image.
- | OUTPUT words for **every** object in the image: multi-label, open vocabulary.
- | FROZEN all weights. The model is a retrieval encoder; it has no tagging head, no classifier, and was never trained for this task.
- | ALLOWED inference-time computation over the model's own outputs, and nothing else.

NO TRAINING

NO SECOND MODEL

NO WORDNET / DICTIONARY

NO REGEX / POS TAGGER

Whatever tagging ability appears must come from [test-time compute of the embedding model](#).

How test-time scaling works on frozen embedding models.

A frozen encoder's inference-time budget takes roughly three forms:

A DEEPER PASS

Read more of the one pass you already ran: every patch vector, not just the pooled one. The retrieval world's **late interaction**.

new information: reclaimed from inside the pass

B MORE PASSES

Run the frozen encoder again on new views: split a document, crop an image, re-encode.

new information: acquired from the input

C BOOTSTRAP

Transform the vectors you already have: whitening, propagation, optimal transport. Where most of the training-free literature lives.

new information: none

Same frozen encoder in all three; they differ only in **what enters the computation**.

Two research questions.

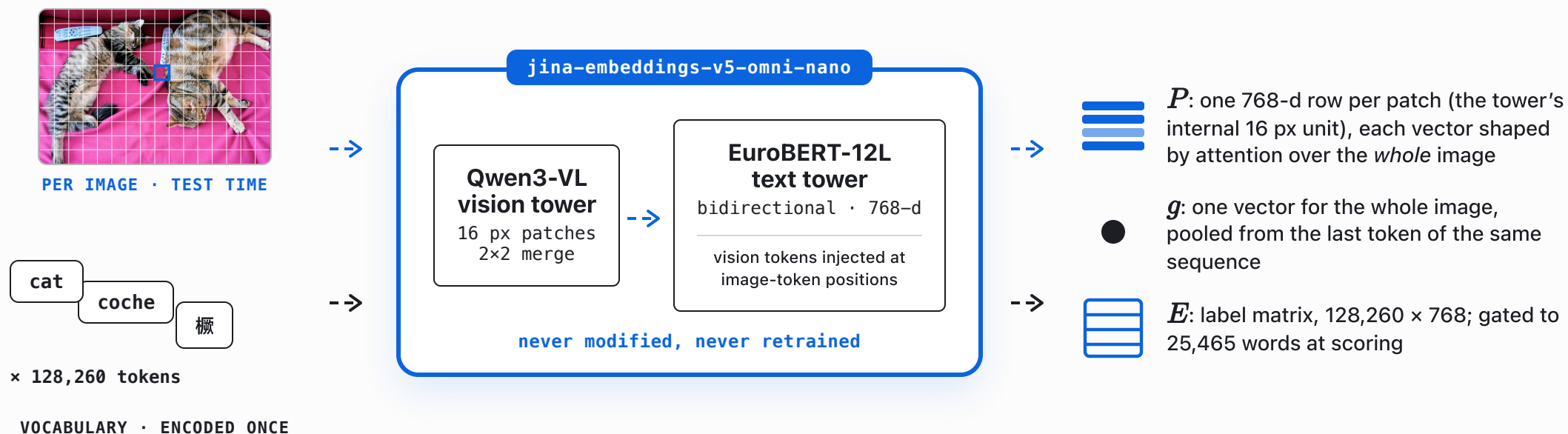
RQ1 Can a frozen embedding model become an image tagger through test-time compute alone?

Prior tagging work pays with training or resources: Tag2Text / RAM / RAM++ train over ~6,400 curated tags; TagCLIP (AAAI'24) and PIAA ("[CLS] is not enough") are training-free but assume a given class list, on two-tower CLIP plus WordNet or POS tools.

RQ2 If it can: which test-time compute scales its accuracy, bootstrapping or new information?

The 2024–26 literature bets on bootstrapping: whitening (PIAA), label propagation (ZLaP), optimal transport (OTTER), Bayesian adaptation (BCA).

Image and text embedding space.



Both paths end in the same space, so $\langle p, e_\ell \rangle$ is well-defined: **any patch of any image, scored against any word the model knows.**

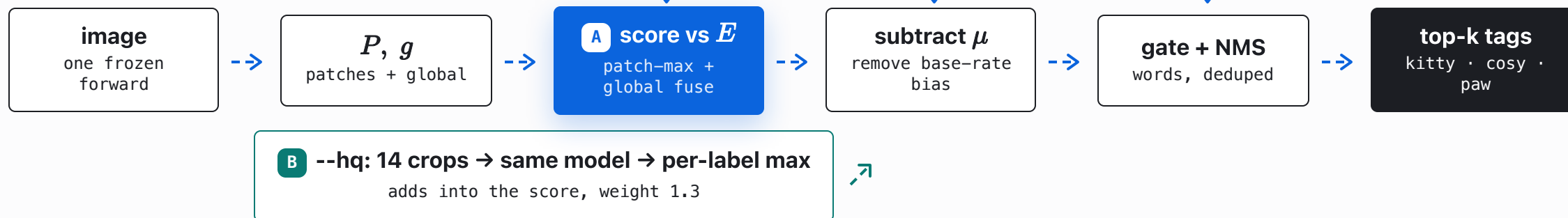
— THE PIPELINE

The complete tagger workflow as test-time compute.

OFFLINE



PER IMAGE



Every box is the frozen model or plain arithmetic: **A** reads more of one pass; **B** runs new passes on new pixels.

The label set is the tokenizer's own vocabulary.

No curated tag list. Take all **128,260** tokens and encode each one as text into a label matrix

$$E = [e_\ell]_{\ell \in V}, \quad e_\ell = \text{encode_text}(\ell) \in \mathbb{R}^{768}, \quad |V| = 128,260$$

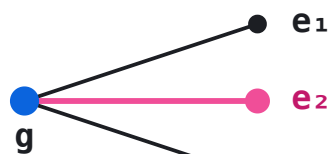
One matmul now scores any image against the entire vocabulary; fragments are filtered later by a word-start gate (step 4).

```
wrong: image_vec · embed_tokens.weight  
       → Tutor, avatar, PyTuple (garbage)  
right: image_vec · encode_text(token)  
       → kitty, cosy, plush, paw (aligned)
```

Why encode_text, not the embedding table? The model has `tie_word_embeddings = false`: the input-embedding table lives in a different space than the pooled output. You must encode each token *as a text string* to put labels in the image's space.

Score every patch.

GLOBAL POOLED

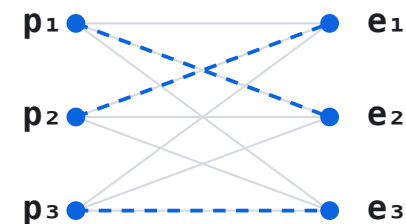


$$s_\ell = \langle g, e_\ell \rangle$$

one vector dominated by the most salient object

mAP 0.264

PER-LABEL MAX OVER PATCHES



$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \langle g, e_\ell \rangle$$

each label keeps its strongest patch match — late interaction, CoBERT-style

mAP 0.635

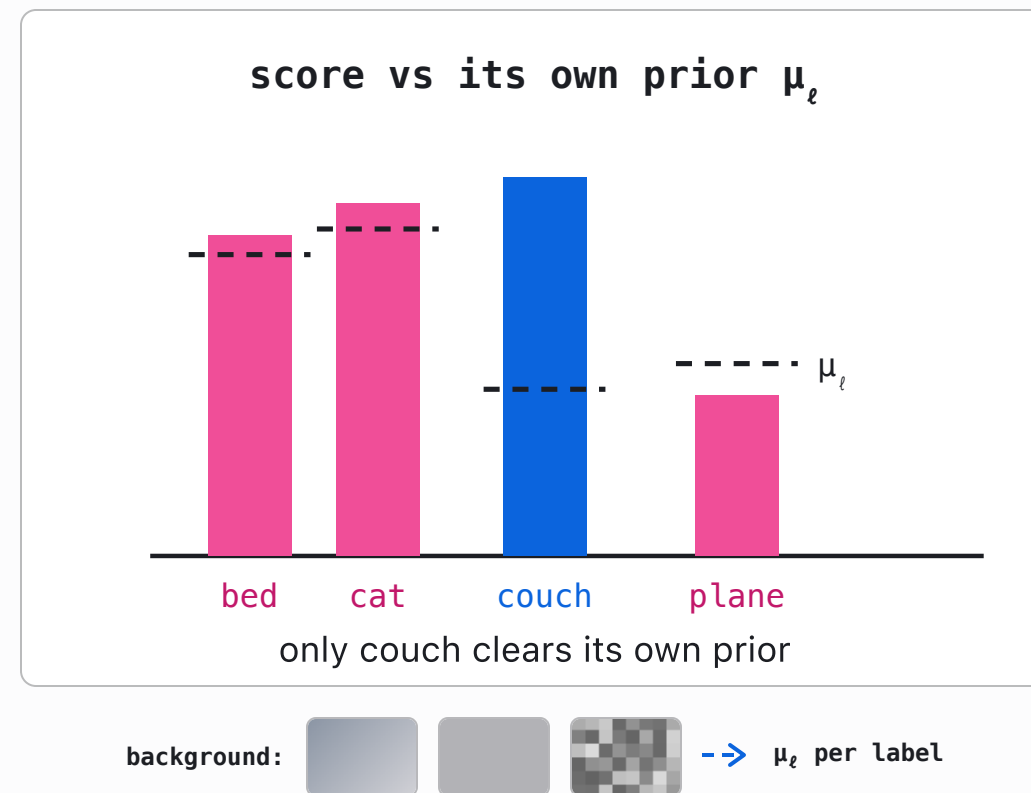
A The patch vectors already exist in the forward pass. Scoring each label against all of them, and keeping the max, is one extra step over outputs you already have.

Subtract the per-label prior.

Raw cosine has a base-rate bias: generic words ("bed", "cat") score high on almost any image because they sit near the image modality. Estimate a per-label prior once from neutral background images, then subtract it:

$$\tilde{s}_\ell = s_\ell - \mu_\ell, \quad \mu_\ell = \mathbb{E}_{x \sim \text{background}} [s_\ell(x)]$$

No calibration set, no labels, one offline pass.



The tokenizer already knows what a word is.

Word-start gate. Byte-BPE tokenizers encode a leading space as the glyph $\hat{G}$; a token beginning with it starts a new word: $\hat{G}$ cat is a whole word, $\hat{G}$ etComponent is a fragment. That one built-in signal is the gate; no external dictionary.

Embedding-NMS. Synonyms and multilingual variants (кот / Cat / кот / cat) collapse into one, using cosine in the model's own space. Walk labels by score; keep ℓ only if $\max_{k \in \text{kept}} \langle e_\ell, e_k \rangle < 0.6$. Again: no lookup table, just the geometry.

raw vocabulary

128,260 tokens

$\hat{G}$ kitty · $\hat{G}$ cat · GetComponent · _cpp · 猫 · quisesites

↓ word-start gate: keep tokens with the leading-space mark $\hat{G}$

whole words

25,465

kitty · cat · kitten · chatte · kitt · cats · kittens · cosy · ...

↓ embedding-NMS: walk down by score; drop any word with cosine ≥ 0.6 to one already kept

distinct concepts

top-k

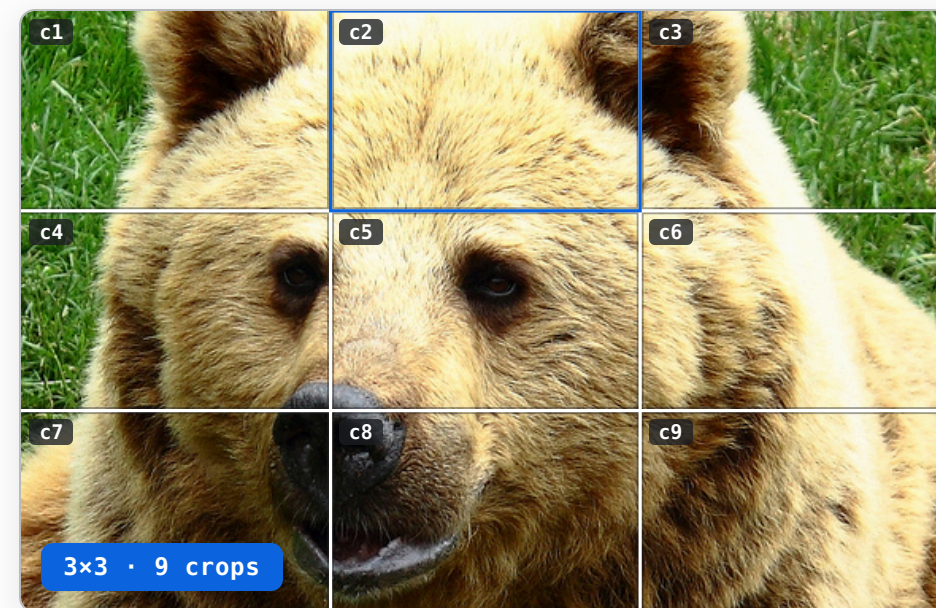
kitty · cat · kitten · chatte · cats · cosy · plush · crib

Multi-crop re-encoding.

Class-Wise Reranking (CWR), the crop-and-rescore idea from TagCLIP (AAAI'24), extended to a full-coverage grid of 14 crops (overlaid right) through the same frozen model. A crop is a *new input image*, itself re-patchified by the tower; a small object fills whichever crop contains it, so weak signal becomes strong.

$$S_\ell = \tilde{s}_\ell + 1.3 \left(\max_{c=1..14} s_{c,\ell} - \mu_\ell \right), \quad s_{c,\ell} = \max \left(\max_{p \in P_c} \langle p, e_\ell \rangle, \langle g_c, e_\ell \rangle \right)$$

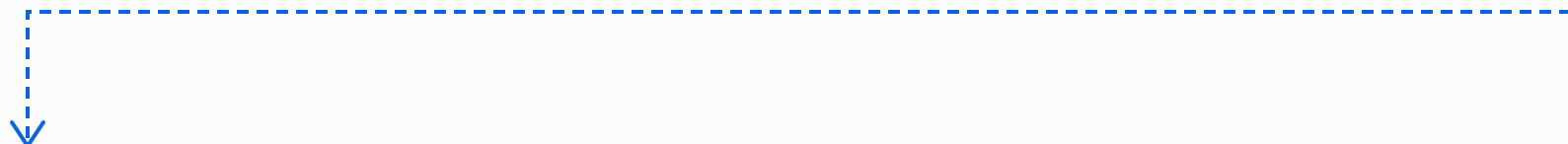
B Per-label **max**, not averaging: **averaging hurt**. The outlier crop is the evidence.



fast: wolf, roar, muzzle → --hq: fur, bear, muzzle
multi-crop re-encoding corrects the single-pass misreading

The entire tagger is one scoring function.

the same score , applied to each crop as a fresh image


$$S(\ell) = \underbrace{0.7 \left(\max_{p \in P} \langle p, e_\ell \rangle - \mu_\ell \right)}_{\text{patch evidence}} + \underbrace{0.3 \left(\langle g, e_\ell \rangle - \mu_\ell^g \right)}_{\text{global context}} + \underbrace{1.3 \left(\max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell \right)}_{\text{multi-crop re-encode}}$$

A Patch evidence: algebra over rows the forward pass already produced. Free. +0.37 mAP.

A Global context: the pooled vector, de-biased by its own background prior. Free.

B Multi-crop: 14 fresh forward passes on new pixels. 14× the cost. +0.075 mAP.

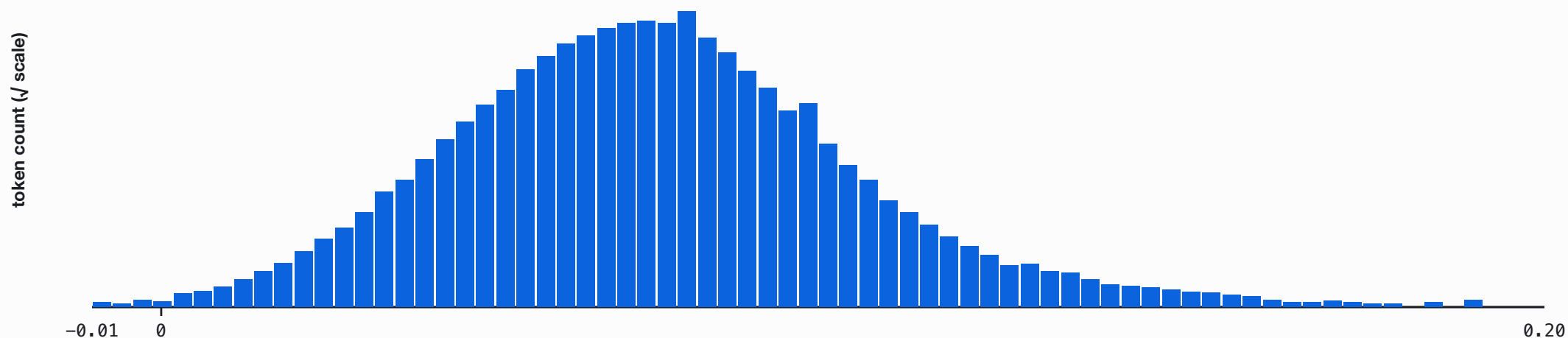
then $\ell \in \text{word-gated vocabulary} \rightarrow \text{embedding-NMS } (\tau = 0.6) \rightarrow \text{top-}k$

No head, no logits, no learned threshold: **three fixed weights, two background priors, and max operations over frozen-model outputs.**

The pipeline sharpens a distribution.

1 · raw fused score

all 128,260 tokens · 0.7 patch-max + 0.3 global



a smooth bell: every token is somewhat close to every image

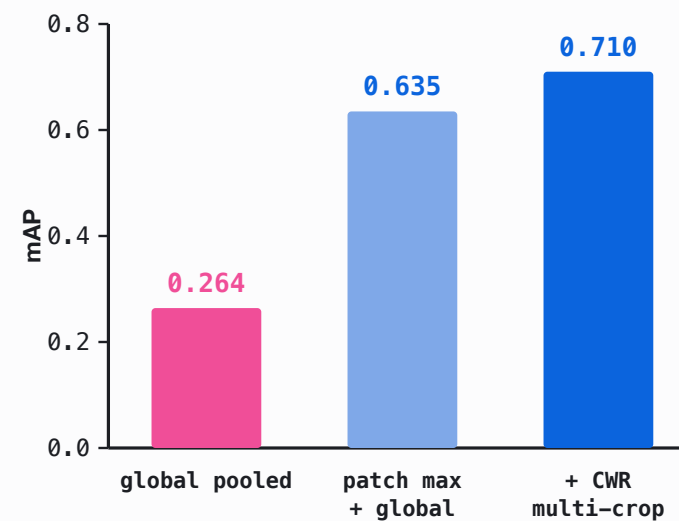
Real pipeline output, one image, all 128,260 tokens. Tagging is distribution sharpening.

RESULTS

Results on COCO-150.

METHOD	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

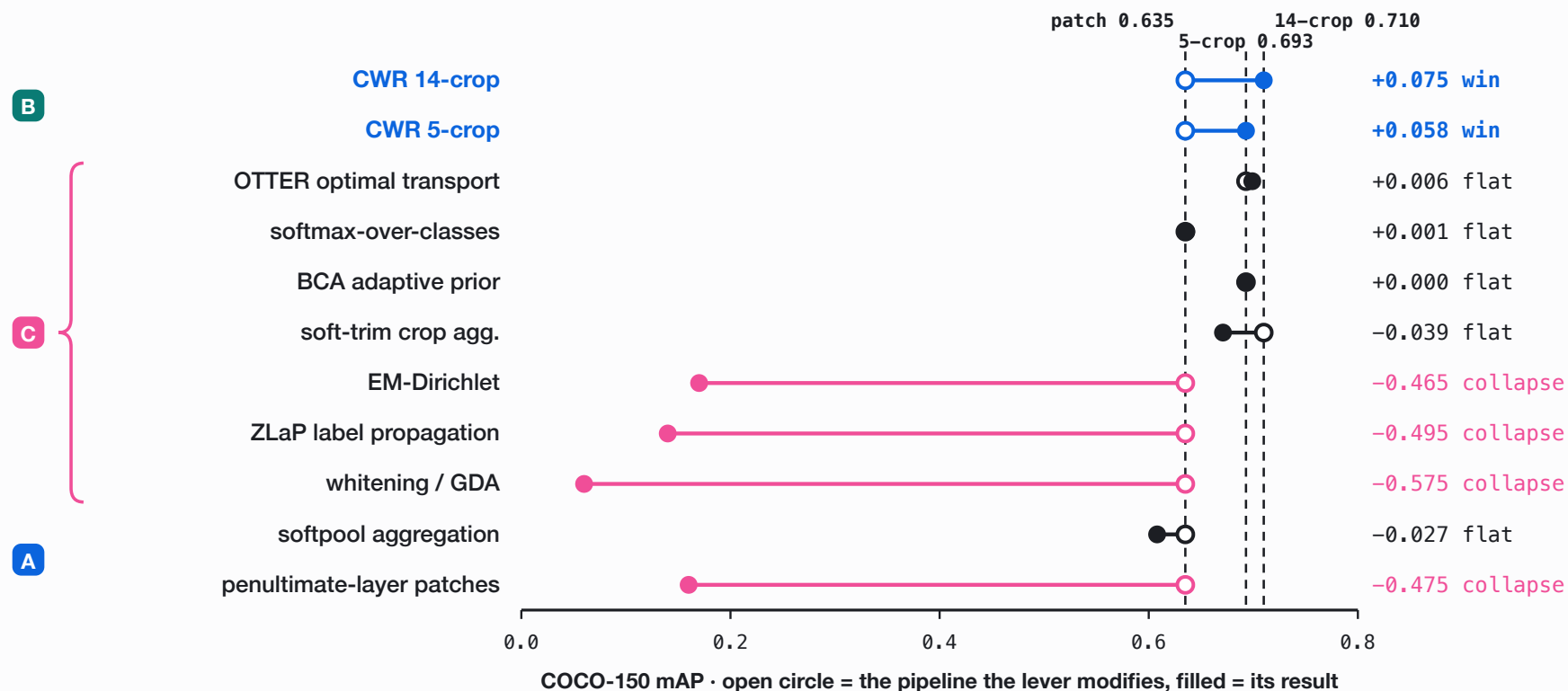
150 COCO-val images, real multi-label ground truth, avg 2.93 labels/image. softpool trades mAP for top-k precision; the ladder tracks the max-pool path.



mAP: global → patch → +CWR

Does bootstrapping scale accuracy at test time?

One example: **OTTER** re-balances scores toward a target class distribution, which only shuffles vectors it already has. Its open circle sits at the 5-crop baseline of **0.693**, and it lands at 0.699.



Only new information moves accuracy.

Bootstrap methods were designed for weakly calibrated, single-label, two-tower CLIP. This encoder's space is already aligned, calibrated, and multi-label: **there is nothing left to recover.**

C BOOTSTRAP

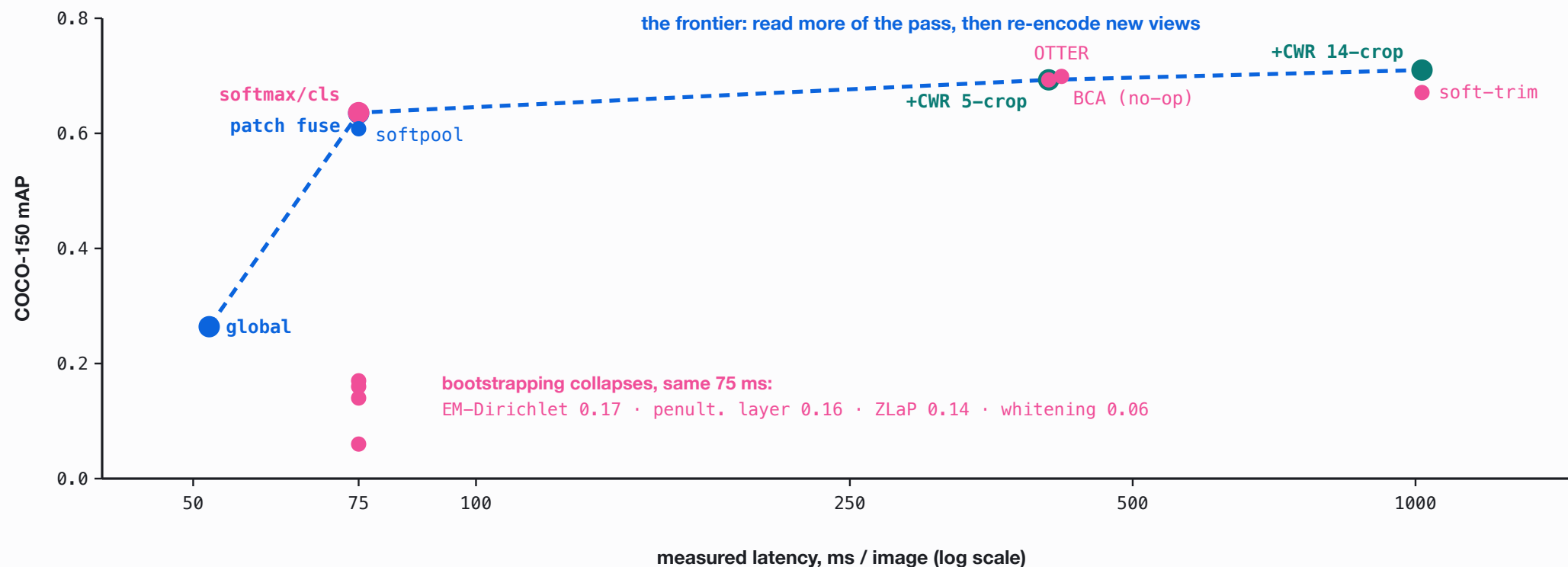
whitening 0.06 · ZLaP 0.14 · EM-Dirichlet 0.17 · OTTER
+0.006 over its own base. None improves the pipeline it
modifies.

B MORE PASSES

CWR crops → mAP 0.710, P@1 0.813. The only intervention
that improves results.

The ceiling is the 1B model itself. Real test-time compute here means
giving the model more to look at, not re-arranging what it already saw.

Accuracy versus test-time compute.



The frontier is traced by reading more of the pass and re-encoding new views;
bootstrap sits at the same cost, at or below it (<0.1 ms/img, measured).

Extend to n-grams.

Every tag so far is a single word. Can more **test-time compute** turn it into a grounded **noun phrase**, with no part-of-speech tagger and no grammar?



ttc tagging
pipeline
--ngram beam
search



n = 1

kitty

cosy

plush

crib

n = 2

couch kitty

black cosy

grey plush

grey crib

n = 3

grey couch kitty

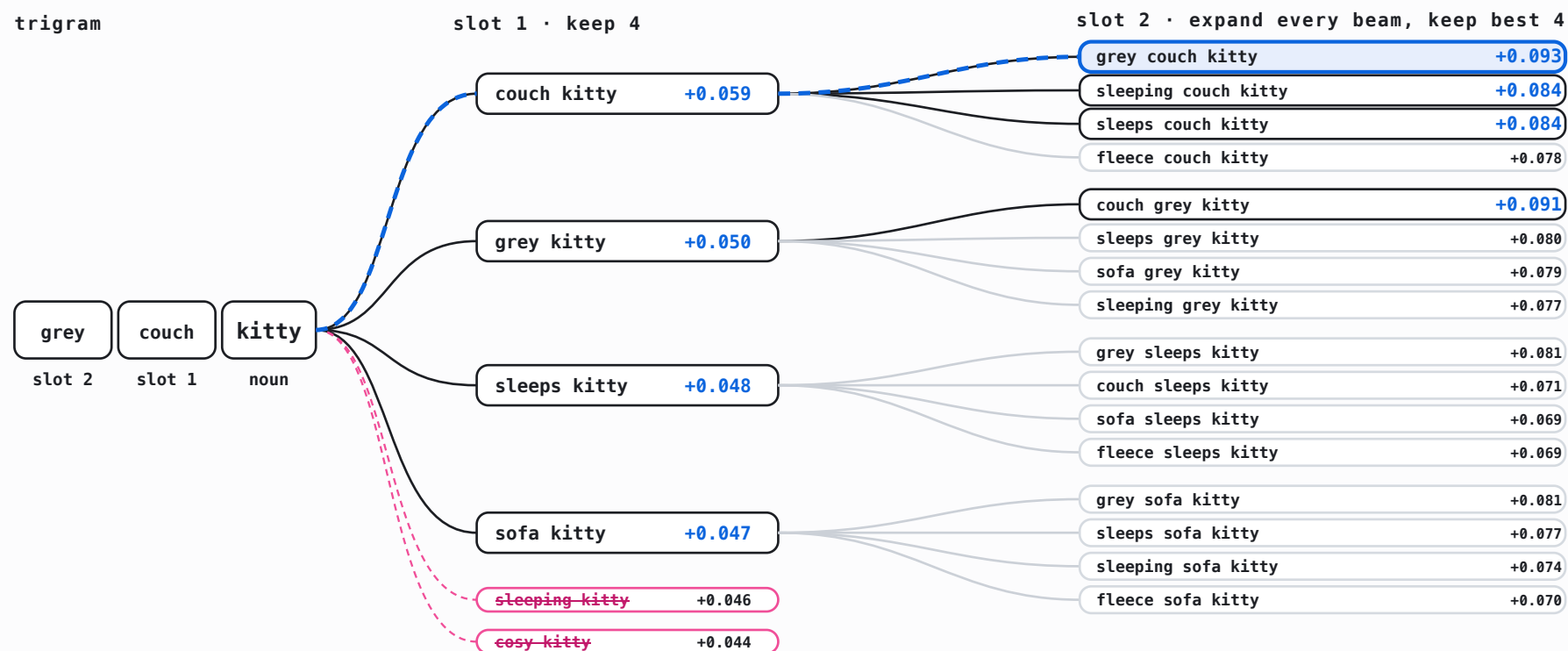
sleeping black cosy

grey sleeping plush

sleeps grey crib

Beam search over n-gram slots.

Each slot is filled one at a time; the encoder scores the full phrase against the region:
 $\langle E(\text{phrase}), \text{region} \rangle - \langle E(\text{kitty}), \text{region} \rangle$. Beam width 4.



The winning path re-orders its own words: "grey couch kitty" beats "couch grey kitty". Word order, resolved by an embedding model with no grammar.

n-gram tagging in action.



DEFAULT onstage attendees livest concert presenter
CROPS onstage venue ceiling projector theater
N=2 conference onstage projector attendees attendees livest projector concert projector presenter
N=3 conference crowd onstage lobby projector attendees projector attendees livest projector lobby concert attendees projector presenter



DEFAULT driv rims ford traction sidew
CROPS carro rims windshield steering beach
N=2 windshield driv windshield rims sail ford windshield traction blue sidew
N=3 blackjack windshield driv dealership windshield rims windshield yacht ford yacht sedan traction blue car sidew



DEFAULT attendees passengers protestors migrants crowded
CROPS attendees commuters demonstrators synagogue crowded
N=2 protester attendees demonstrators passengers passengers protestors attendees migrants demonstrators crowded
N=3 passengers protester attendees crowded demonstrators passengers passenger attendees protestors passengers protester migrants
passenger demonstrators crowded

Default, crop re-encoding, beam-searched n-grams. Top-5 per mode, unstaged photos. [Verbatim](#), [unfiltered](#).

Versus the recent tagging literature.

LINE OF WORK	THEIR SETUP	OUR DELTA
Tag2Text / RAM / RAM++	trained tagging models, curated ~6.4k tag list	training-free, label space = tokenizer vocab
TagCLIP	CLIP two-tower, penultimate layer, DMAR+CWR	single omni model, last layer is the output space
PIAA	patch-level scoring ("[CLS] is not enough") + GDA whitening	patch thesis confirmed (+0.37 mAP); whitening collapses (0.06)
recent calibration (OTTER, BCA)	calibration set or target prior to de-bias	one offline background prior, zero calibration

Both questions come out the way we hoped. A frozen embedding model does become a tagger with no training, and the only compute that scales it is the kind that adds new information.

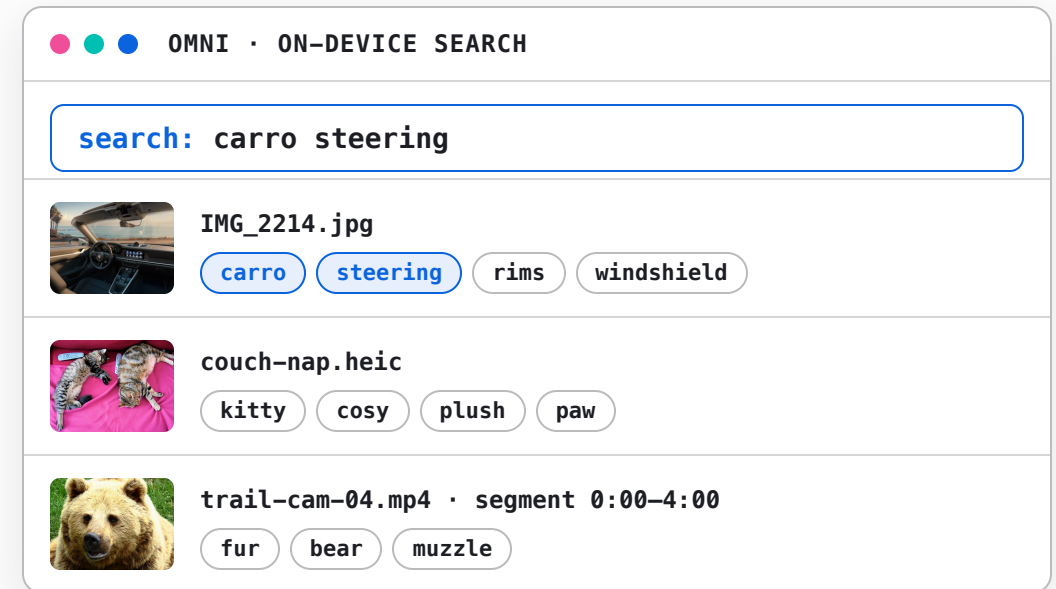


DEPLOYMENT

On-device deployment in Omni.

The Python study became a feature of **Omni**, a native on-device semantic-search app (Swift + MLX, same frozen model). The port reuses the architecture directly:

SAME FORWARD PASS	the patch rows already exist for the file's <i>embedding</i> ; tagging adds one matmul.
~0.6 MS / IMAGE	about 4% overhead on the embed step. Tagging is effectively free at index time.
TAGS = SNIPPETS	tags land in the search snippet: media becomes <i>keyword</i> -searchable.
SELF-CALIBRATING	the background prior μ is estimated on-device from the first 64 images it sees, then frozen.



tag chips as they appear in results; the third row is a video segment

Images, video, and scanned documents.



IMAGE

Tagged at **index time**, inside the embedding forward. Patch-max + global, prior-centered, gated, NMS-deduped.



HQ IMAGE

A background **re-tag pass** adds the production CWR variant: 5 crops, per-label max.



VIDEO

32 frames per **240-second segment**, one sequence through the tower: patch-max pools over **space and time**.



SCANNED PDF

Pages render to images, same path: **per-page tags** ("invoice, table, signature") as the snippet.

Test-time compute for embedding models.

A frozen embedding model can do more than its training reveals. You reach the rest by spending compute at inference, not by adding parameters. Both of my talks push on that same idea.

AIE SF TALK

Spend test-time compute to get more **relevance**, better retrieval on the task the encoder was already trained for.

THIS TALK

Spend test-time compute to get a new **capability**, image tagging, a task the encoder was never trained for.

Scaling test-time compute of an embedding model **unlocks tasks beyond retrieval.**

Han Xiao

VP of AI, Elastic

[@hxiao](#) · [in/hxiao87](#)

GITHUB REPO

